This gives you two helpful command-line tags: rspec --only-failures and rspec --next-failure. The --only-failures tag runs all the specs that failed the last time they were run (regardless of whether that run was the last time you ran any RSpec specs). The --next-failure tag does the same thing, but it stops the run once one of the specs in the list fails again.

I use these two commands all the time. In conjunction with Spring, they let me run exactly the specs I'm working on very quickly.

Prescription 45	Use rspec --only-failures and rspec --next-failure to only run failing specs, which allows you to focus on failures easily.

Minitest

If you're invoking Minitest through the rake tasks that Rails provides, then it's easy to run a single file's worth of output just by passing the file name at the command line:

```
$ rake test test/models/task_test.rb
```

The argument can be an individual file or a directory. You can have more than one file or directory if they're separated by spaces.

This behavior is relatively new to Rails. If you've been using Rails for a while, you may know that it provides a number of testing subtasks, such as rake test:models, that wrap the expected directories and run all the tests in that directory. Since Rails has been nice enough to add the directory behavior separately, I recommend using the actual directories rather than trying to remember the names of all the rake tasks.

It's a bit more involved if you want to run a single individual test rather than an entire file. As far as I can tell, there's no way to do that directly from the Rails rake task, so you need to drop down to the Ruby command line. The invocation looks like this:

```
$ ruby -Ilib:test test/models/task_test.rb -n test_a_completed_task_is_complete
Running tests:
.
Finished tests in 0.019890s, 50.2765 tests/s, 100.5530 assertions/s.
1 tests, 2 assertions, 0 failures, 0 errors, 0 skips
```

In this command, you're running ruby with the capitol *I* option, which specifies directories to add to the load path. Specifically, you need the lib and test directories so that you can find the test_helper file your test is going to require. (You may be able to get away with not adding lib.) Next is the name of the file, and